

Recursion, Fractals and space-filling curves in Logo

Alina Musaeva

Julian Schwarzer

Lars Wittner

June 2026

Abstract

This paper introduces Logo as a functional programming language, focusing on recursion as the basis for fractals. These are key concepts for grasping how Logo can be utilised to generate space-filling curves. We will also explain L-systems and the fundamental mathematics behind them. We will go through the base commands and view a few examples of recursion, as well as explaining how to create fractals in different ways. Topics covered will include edge- and node-rewriting.

1 Introduction

In chapter Section 2 we will explore what Logo is, its main features and how to work with it. We will show you some base commands which are required to better understand it in other chapters. In chapter Section 3 and Section 4, we will methodically explore the concepts of recursion and fractals using the Pythagoras tree, Koch's flake. Finally, we will cover topics such as L-system, edge- and node-rewriting and branching in chapter Section 5. This will enable you to understand the idea of space-filling curves in Logo.

2 Definition of Logo and its origin

Before we move on to recursion and fractals, we're going to find out what Logo is and learn some base commands.

Logo is a programming language that is well-known for its use of turtle graphics. With Logo, you can draw different figure shapes.

It was invented in 1966 in Cambridge (USA) by Wally Feurzeug, Seymour Papert, Daniel Bobrow and Cynthia Solomon.

The original intention behind the creation of this language was to provide a means for children in schools to learn how to code and enhance their mathematical skills [8].

2.1 Turtle Graphics

The turtle is a pen or a cursor (often represented as a triangle) that moves after a command and draws when the pen is down. You can change the position, direction, pen state, color etc. It will all be displayed on the computer screen [6].

2.2 Data Types

In Logo, data types are dynamic. This means that they do not need to be defined explicitly.

There are four main data types: word, number, list and boolean [3].

1. *word* is used for a sequence of characters
2. *number* for such numbers as integer, float, double etc.
3. *list* is a list of numbers or characters (which may for example be a sentence or an array).
4. *bool* is for true or false

2.3 Base commands

The Logo syntax is very intuitive. It consists of simple words, with or without parameters (Figure 1).

A shortcut is also available for each command, thus avoiding the need to type the entire word each time.

Here are the base commands [4]:

```
FORWARD/BACK steps
RIGHT/LEFT degree
CLEARSCREEN
PENUP/PENDOWN
REPEAT
PRINT [The sentence]
```

As demonstrated in Figure 1, the turtle was moved forward 40 steps, rotated 110 degrees, and moved forward again 49 steps.



Figure 1: command forward and right in ucbLogo

The others commands for instance `CLEARSCREEN` or simply `CS` is used for cleaning the drawings on the display (not the code).

`PENUP` is used to move the turtles position without drawing and `PENDOWN` to return to the drawing mode.

`REPEAT` allows commands to be repeated multiple times, thus saving time.

For example we can draw a square using following command:

```
1 REPEAT 4 [fd 90 rt 90]
```

logo

To print words, numbers or lists we use the following syntax in logo:

Listing 1: Examples of printing

```
1 Print "word
2 Print 8
3 Print [Hello world]
4 Print [1 2 4]
```

logo

Listing 2: Output of Listing 1

```
1 word
2 8
3 Hello world
4 1 2 4
```

logo

2.4 Procedures

The following steps are required to create a procedure (function) in Logo:

Enter *To* followed by the name of the procedure and *End*. The commands should then be entered between these two items. Use parameters if required.

```
1 To ProcedureName :parameter1 ... :parameterN
2   commands
3 End
```

logo

For example, we can now create a procedure that enables us to draw different polygons.

```
1 To PolygonMaker :num1 :len
2   repeat :num1 [ fd :len rt 360/:num1]
3 End
```

logo

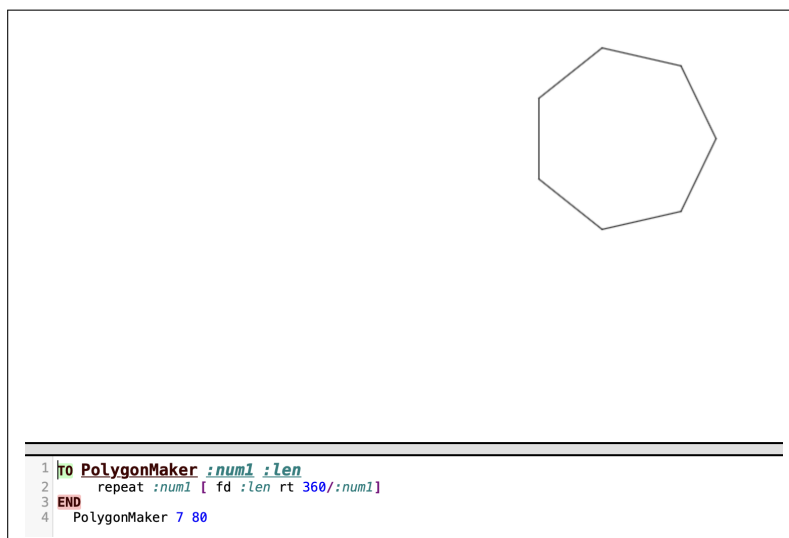


Figure 2: Call of procedure PolygonMaker

In this procedure we used 2 parameters *num1* - number of angles and *len* - length of the sides. So now we can use it to make a triangle, a square, an octagon or even a circle. Besides, this function can be called multiple times.

2.5 Loops

Also a very important part of almost every programming language are loops and if else statement.

If/Ifelse statement

```
1 IFELSE condition [condition true][condition false]
2 IF condition [condition true]
```

logo

Logo is very intuitive. We enter *ifelse* and a condition that will be proven to be either true or false. The corresponding command will then be called. Or simply use *if* if you only have one if-case [3].

While-loop

```
1 WHILE [condition]
2 [commands]
```

logo

The while loop is also pretty simple. First, we enter *While*, then the condition. After that, we enter the commands that will be executed while the statement is true.

For-loop

```
1 FOR [var start end step]
2 [commands]
```

logo

The For loop is slightly different to that in other programming languages. First, we type *For* and then four parameters in braces: the name of the variable, the starting value, the ending value, and the step value. After this, we can enter commands that will be executed within the loop.

Listing 3: Example of using For loop

```
1 for [i 1 10 2]
2 [print :i]
```

logo

Listing 4: Output of Listing 3

```
1 1
2 3
3 5
4 7
5 9
```

logo

3 Recursion

We are now capable of writing programs with these base commands but we still have to introduce recursion for a functional programming language like Logo.

3.1 Definition

Recursion has its origin in the latin word *recurrere* (*to go back*). A recursive written function has to contain three key parts:

1. Base case
2. Calling itself
3. Recursive step

The base case will be a condition that indicates what case our recursive calls will end up terminating. We need this due to our limitations of storage and to avoid an infinite number of calls on our call stack. To actually reach this base case we need a recursive step that needs to be provided by our recursive call that approaches our base case [5].

Fibonacci numbers

```
1 private static int fib(int n) {  
2     if (n <= 0) return -1; // wrong input  
3     if (n == 1) return 0; // base case 1  
4     if (n == 2) return 1; // base case 2  
5     return fib(n - 1) + fib(n - 2); // recursive call  
6 }
```

java

In this java implementation of the Fibonacci numbers (Section 3.1) you can see that there are also multiple base cases possible. For this example the first Fibonacci number is defined to be "0" and the second one has to be "1". After these base cases the recursive call and its embedded recursive steps will give us our final result of our requested n th number. As you might have noticed the recursive step is not only one time but two times in this example. This is because the Fibonacci numbers are defined as the sum of the previous two numbers.

It is very important that the recursive step actually reaches our base case. In this example each reduction ($n - 1$ and $n - 2$) will either reach our first or our second base case and terminate. If we had skipped our base case and had not filtered wrong inputs by $n \leq 0$ the method would not terminate and the recursion will end up in an infinite loop.

The phrase *depth of recursion* determines the amount of nested recursive functions inside of itself.

Recursion often finds usage in science like mathematics *eg. factorials, Fibonacci* or computer science like in functional programming languages like Logo that profit from reduced lines of code and readability.

3.2 Pythagoras tree

The pythagoras tree is a very common example to explain recursion [10]. Its main shape is a square with recursively placed squares on top of the base square rotated by a predefined angle that is calculated based on the famous Pythagoras formula (Figure 3).

To implement this, we will use the previously explained scheme: We declare our procedure within the first line (keyword *to*) with its name being *pythagoras* and two needed parameters (indicated by the colons) as *length* and *depth*. We end our definition with the keyword *end* in our last line.

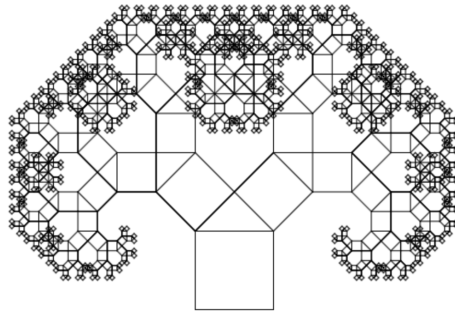


Figure 3: Pythagoras tree

```

1  to pythagoras :length :depth
2      ...
3  end

```

logo

First, we implement our main shape (the square) by repeating it four times going forward and turning right.

```

1  repeat 4 [fd :length rt 90]

```

logo

Now we write our base case, which means terminating our program at depth=0. The depth represents the maximum number of squares placed on top of each other (*recursion depth*).

```

1  if :depth = 0 [stop]

```

logo

We end up at our starting position at the left bottom corner of the just drawn square. To draw our recursive squares, we need to correct our position.

For that we have to reach the top left corner and rotating to the left by 45 degree, which will be our predefined rotation, so both squares will have the exact same length and our Pythagoras tree becomes axially symmetric.

```

1  fd :length lt 45

```

logo

Our recursive squares are then drawn with a reduced length in the scale of the Pythagoras formula and a depth that decreases by "1", which ensures the base case of depth=0 being reached at some point.

```

1  pythagoras :length / (sqrt 2) :depth - 1

```

logo

Because we are using the 45 degree angle, the following derivation explains the new length: We assume our hypotenuse as a and our two catheti as length b .

According to Pythagoras:

$$\begin{aligned}
 a^2 + a^2 &= b^2 \\
 2a^2 &= b^2 \\
 \sqrt{2}a &= b \\
 b &= \frac{a}{\sqrt{2}}
 \end{aligned}$$

We have to change our position again to start the second recursive square.

```
1  rt 90 fd :length / (sqrt 2)
```

logo

Positioned correctly, we perform the same recursive call as our first part.

```
1  pythagoras :length / (sqrt 2) :depth - 1
```

logo

After each recursive depth, we return to the position of the previous one ($depth + 1$).

```
1  bk :length / (sqrt 2) lt 45 bk :length
```

logo

The result of our final procedure will be as follows:

Pythagoras tree

```
1  to pythagoras :length :depth ; variable length and depth
2  repeat 4 [fd :length rt 90] ; square
3  if :depth = 0 [stop] ; base case
4  fd :length lt 45 ; change pos
5  pythagoras :length / (sqrt 2) :depth - 1 ; recursive left
6  rt 90 fd :length / (sqrt 2) ; change pos
7  pythagoras :length / (sqrt 2) :depth - 1 ; recursive right
8  bk :length / (sqrt 2) lt 45 bk :length ; go back
9  end
```

logo

4 Fractals

4.1 Definition

To understand the possibilities of graphical approximation Logo has, we are going to introduce the concept of fractals. Fractals are the idea of performing a recursion with a *recursion depth* of infinity and are self-similar which means they are made of smaller copies of itself. You can imagine them by "breaking" (*lat. fracere*) them apart, but you will not be able to tell the difference between their parts [1].

Fractals are a theoretical concept that cannot be used in the sense of what they mean due to physical limitations of the real world like the finite resolution a monitor has. Some well-known fractals from nature are the golden ratio (Figure 4) or the Sierpiński triangle (Figure 5) which was considered a purely mathematical concept for a long time. However, this form also occurs in the form of the enzyme citrate synthase [2].

In order to be able to work with fractals nonetheless, we can use visualize by recursion with a relatively high recursion depth. The depth of recursion is the scale of sharpness fractal images have.

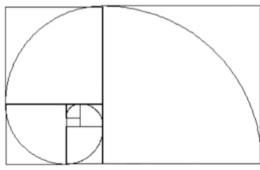


Figure 4: Golden ratio

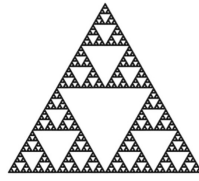


Figure 5: Sierpiński triangle

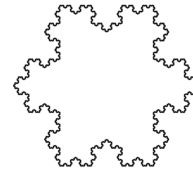


Figure 6: Koch flake

4.2 Koch flake

The Koch flake became quite popular for its simplistic form and minimal lines of code for a quite nice looking shape of a snowflake. First, described by Helge von Koch in 1904. It is made up of three triangular shaped Koch curves which follow simple rules [9].

At first, a Koch curve is simply a line with a predefined length. By increasing the depth, Koch curves will be placed on top of these lines forming a triangular shaped spike on top of the base line. Depth=0 is a straight line and the higher the depth, the more triangles are placed on top of each other.

We will declare our Koch curve in the same way we did with the Pythagoras tree.

```
1  to curve :length :depth
2      ...
3  end
```

logo

We will start by defining our termination condition (*base case*) by drawing a straight line.

Part 1/4:

```
1  if :depth = 0 [fd :length stop]
```

logo

To achieve our Koch curve with triangular placement, it consists of four parts:

The first quarter features a straight stretch that spans one-third of the total length. The second and third quarters represent the rotation, with both sides of the triangle placed at a depth on top. To finalise the curve, we perform the same process as in step one, covering a third of the length.

```
1  curve :length / 3 :depth - 1
2  lt 60
3  curve :length / 3 :depth - 1
4  rt 120
5  curve :length / 3 :depth - 1
6  lt 60
7  curve :length / 3 :depth - 1
```

logo

To get a snowflake-shaped Koch flake we simply need to expend the Koch curve by another small procedure *flake*. This flake repeats our *curve* three times and rotates 60 degree between each curve, so we end up having the shape of a triangle (??).

Koch flake

```
1  to curve :length :depth
2    forward and stop
3    if :depth = 0 [fd :length stop]
4    curve :length / 3 :depth - 1
5    lt 60
6    curve :length / 3 :depth - 1
7    rt 120
8    curve :length / 3 :depth - 1
9    lt 60
10   curve :length / 3 :depth - 1
11  end
12
13  to flake :depth
14    repeat 3 [ curve 200 :depth rt 120 ]
15  end
```

logo

5 Lindenmayer-systems

The Lindenmayer-system or short the L-system was invented by Aristid Lindenmayer in 1968. He was a Hungarian biologist who wanted to recreate plant growth as accurately as possible.

For this purpose he came up with this (V, ω, P) . V is the alphabet, which consists of non-terminal and terminal symbols. Non-terminal symbols are symbols that will be replaced eventually, whereas terminal symbols will not be replaced at all. ω is a non-empty starting word of an L-system which will be used to form new words that will be part of the L-system. These words are created with production rules P [7].

5.1 Rewriting Methods

This paper will only discuss the deterministic and context-free L-system. We start with an example to get the basic concept of these systems.

When looking at $(\{a, b\}, a, P)$ where the production rule P describes $a \rightarrow ab$ which means that each time a word contains the letter a , it is replaced in the next step for ab and $b \rightarrow bb$ which means that every b will be replaced for bb . We start with the letter a because our starting word $\omega = a$ in our system a is replaced by ab . Now we have two letters in our word and each of them will be replaced simultaneously. This is a big difference compared to normal context-free languages where only one symbol is replaced at a time. ab becomes $abbb$ because a becomes ab and b turns into bb . Next, $abbb$ turns into $abbbbbbb$ (Figure 7).

We will discuss two ways of creating an L-system using turtle graphics. They are edge-rewriting and node-rewriting. In these rewriting systems the alphabet $V = \{+, -, F\}$. These characters form words that can be interpreted in the turtle graphics. $+$ means turning the head of the turtle to the right and $-$ means turning to the left, both with a predefined angle that does not change in one system. and F stands for going forward. The production rules in edge-rewriting replace the F , the edges for a new shape. The edges in node-rewriting do not get replaced, there are new symbols $A \notin V$ that have no interpretation in the turtle-graphic. Their only purpose is to be replaced [7].

$$\begin{aligned}
a &\rightarrow ab \\
ab &\rightarrow abbb \\
abbb &\rightarrow abbbbbb
\end{aligned}$$

Figure 7: Example of words in a deterministic and context-free L-system.

5.1.1 Edge-rewriting

Edge-rewriting is the more intuitive way of creating L-systems. For example, Figure 8 shows an L-system $(\{+, -, F\}, F, P)$ with $P := F \rightarrow +F - F$ and an angle $\delta = 45^\circ$. As mentioned earlier, L-systems have terminal symbols and non-terminal symbols. In our example $\{+, -\}$ are terminal symbols, they do not get replaced by our production rules and F is a non-terminal symbol since we have a rule for replacing F with something else. However, our F already had a meaning being an edge in our L-system. This edge in our example (Figure 8 image 1.) (depth=0) is replaced by a shape (Figure 8 image 2.) (depth=1).

There is also the possibility to use two F s, for example having now F_r and F_l for more variation and complexity in our L-system. In Figure 8 we see the difference between one rule in image 3.1 and two rules in image 3.2, both with depth=2. We define $P := F_r \rightarrow +F_r - F_l$, $F_l \rightarrow -F_r + F_l$ and the starting word $\omega = F_r$. The first rule is the same rule as in the earlier example and our second rule is the same shape to replace our lines but mirrored to the first rule. As seen in Figure 8 image 3.2, with two reproduction rules, the image looks more complex [7].



Figure 8: Example of L-system using edge-rewriting.

5.1.2 Node-rewriting

For Node-rewriting we do not replace our edges F anymore. We introduce new symbols $\{A, B, C, \dots\}$ that do not have any meaning in our system so far. We will only use the symbol A in our example (Figure 9).

Our new L-system $(\{+, -, F, A\}, FA, P)$ with $P := A \rightarrow +AF - AF$ and an angle $\delta = 45^\circ$. Since the non-terminal symbols do not have any meaning, they do not change the image that we get by illustrating our L-system, but they determine how the next step of the recursion will look like. As you can see in Figure 9, the edge in image 1 (depth=0) stays in image 2 (depth=1), the edges in image 2. also stay in image 3.1 (depth=2). Just like with edge-rewriting, we can create L-systems with multiple rules. We define $P := A_r \rightarrow +A_r F - A_l F$, $A_l \rightarrow -A_r F + A_l F$ and $\omega = FA_r$. The first two recursion steps look similar to the L-system with only one rule however in our example they differ with depth=2 in Figure 9 image 3.1 with one rule and image 3.2 with two rules.

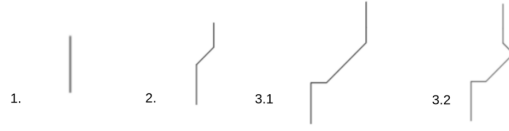


Figure 9: Example of L-system using node-rewriting.

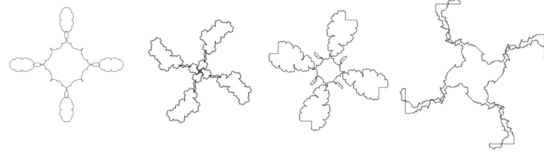


Figure 10: Designs created with our examples and a square.

5.2 Designs

Designs are essentially a repeated pattern of our fractal. To make our fractals look even more fabulous, the next step is to use designs. Designs are essentially a repeated pattern of our fractal. As we saw earlier with the Koch flake, this involves repeating the Koch curve three times. The first step is to choose a polygon on which to base the rewriting. In our example, we will choose a square. With our square, we now have four edges as a basis, whereas we only had one edge in our fractal. For edge rewriting, instead of replacing one line, we now have four lines, each of which is replaced by our shape. I. In fig. 10, we can see that the two designs on the left were created using edge rewriting: the one on the left was created using one rule and the one on the right was created using two rules. We can do the same for node rewriting by placing nodes at the end of each edge, ensuring that every corner has a node. Each node is replaced by our rule for a shape. In our example, the two designs on the right in Figure 10 were created using node rewriting. The design on the left was created using one rule and the design on the right was created using two rules. Whenever we replace edges or nodes in any polygon, the line always stops where it began (Figure 10). All the designs in Figure 10 have depth=10.

5.3 Branching

Using edge-/node-rewriting, we can now create many L-systems. By using the turtle graphics, we can draw any number of angles and lines like the Pythagoras tree, the Koch flake or any self made L-system lines or designs. But they will always only be a line. However, the world of plants is nothing but a world of branching. Lindenmayer was a biologist and he wanted to recreate the world of plants. This can be achieved using either edge- or node-rewriting. We will only discuss it with edge-rewriting, but it works the same way for node-rewriting.

5.3.1 Stack

First, we need to know what a stack is and how it works for us to use branching.

A stack is a data structure that follows the principle of Last in First out or "LiFo". Two functions are used to operate the stack. The first function is *push(input)*, when calling it, *input* will be written in our stack. The second function is *pop()*. It does not get any parameters and returns the last thing that was written on the stack removing it at the same time.



Figure 11: Example of branching with edge-rewriting.

5.3.2 Edge-rewriting with a Stack

We will introduce two new symbols in our alphabet. We define $[:= \text{push}(\text{position}, \text{direction})$ and $] := \text{pop}()$. Every time we use $[$ in a word of our system, we save the current position and the direction the turtle is facing, and every time we read $]$ in our word the turtle returns to that position and the direction faced when we pushed that position. In our example we use the L-system $(\{-, +, F, [,]\}, F, P)$ with $P := F \rightarrow F[-F][+F]F$ and $\delta = 45^\circ$. We see that there are three branches, with depth=1 (Figure 11 image 2). In Figure 11 image 3 (depth=2) we can see why the stack is important and why we could not use a list or other data structures without "LiFo". The word for our 3. image is $F[-F][+F]F[-F[-F][+F]F][+F[-F][+F]F]F[-F][+F]F$ and when we interleave the braces as we did in this example, we must ensure that the order of the returns is not mixed up, because if it were, our example would consist of three parts, which would mean that it would no longer be a fractal.

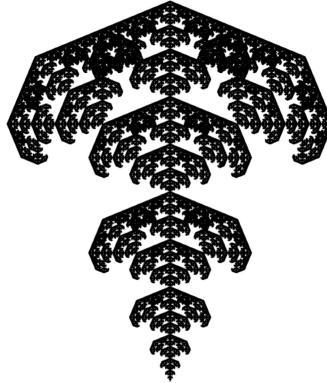


Figure 12: Example of edge-rewriting with a stack

In Figure 12 we can see our example with depth=9. Now we have created a fractal that resembles a plant, and we can create many more, so we can now recreate a world of plants using branching.

6 Conclusion

We have learned some basic commands and data types, as well as how to create procedures in Logo. Furthermore, we now understand the key features that define recursion as a concept, and we have grasped the theoretical idea of fractals. Finally, we have learned how to create our own fractals. Edge-rewriting is more intuitive than node-rewriting, and using a stack enables us to create fractals that resemble plants.

References

- [1] Manuel Alfonseca and Alfonso Ortega. “Representation of fractal curves by means of L systems”. In: *Proceedings of the conference on designing the future*. 1996, pp. 13–21.
- [2] anonym. *Erstes fraktales Molekül in der Natur entdeckt*. 2024.
- [3] anonym. *Logo Language Guide* — *calormen.com*. <https://www.calormen.com/jslogo/language.html>. [Accessed 30-06-2026].
- [4] Brian Harvey. *Computer science logo style: Symbolic computing*. Vol. 1. MIT press, 1997.
- [5] *Introduction to Recursion*. [Accessed 01-07-2026]. 2026.
- [6] Henry Lieberman. “The TV Turtle a Logo graphics system for raster displays”. In: *The papers of the ACM Symposium on Graphic Languages*. 1976, pp. 66–72.
- [7] Przemyslaw Prusinkiewicz and Aristid Lindenmayer. *The algorithmic beauty of plants*. Springer Science & Business Media, 2012.
- [8] Cynthia Solomon et al. “History of logo”. In: *Proceedings of the ACM on Programming Languages* 4.HOPL (2020), pp. 1–66.
- [9] Eric W. Weisstein. “Koch Snowflake.” *From MathWorld—A Wolfram Resource*. [Accessed 01-07-2026].
- [10] Eric W. Weisstein. “Pythagoras Tree.” *From MathWorld—A Wolfram Resource*. [Accessed 01-07-2026].